Hindsight: Giving an On-Call Agent a Graph for a Brain

Built for Graph Hacks by WeMakeDevs and FalkorDB. Tracks: Agents That Act on Connected Data, Agent Memory and Coordination, Company Brain.

Repo: [https://github.com/Aishwary2004Gupta/Graph-Hacks]

Video: [https://youtu.be/ULE3_ogimlY]

The 3 a.m. question

It's 3 a.m. and checkout is failing. Within a minute you need answers to four questions:

1. What changed recently?
2. What else breaks if this service stays down?
3. Who has fixed something like this before?
4. Why was it built this way in the first place?

None of those answers lives in one record. The recent deploy is in your CI system. The dependencies are in a service catalog. The people who fixed the last outage are in a postmortem. The reason for the design is in an RFC from months ago. The answer is the *connection* between all of them.

That is the problem I built Hindsight to solve, and it is the reason I used a graph rather than a pile of documents.

Why not just vector search?

Embedding search is good at "find text that looks like this question." It is weak at "follow this chain across four kinds of things." Ask a vector store what breaks if `postgres-main` goes down and you get paragraphs that mention Postgres. You do not get the set of services that depend on it two and three hops away, ranked by distance.

In a graph that is a single query:

```
MATCH p=(d:Service)-[:DEPENDS_ON*1..4]->(s:Service {name:$service})
RETURN d.name AS service, d.tier AS tier, min(length(p)) AS hops
ORDER BY hops, tier
```

The agent does not guess the relationships. It reads them.

The architecture

Hindsight uses three kinds of graph, all in one FalkorDB instance:

- **`brain_<tenant>`** is the company knowledge graph. It holds services, teams, people, deploys, incidents, runbooks, decisions and documents, connected by relationships like `DEPENDS_ON`, `DEPLOYED_TO`, `CAUSED_BY`, `RESOLVED` and `SOURCED_FROM`.
- **`mem_<tenant>_<user>`** is private memory for each user: episodes (what was asked), entities, and facts that can be superseded.
- **`shared_<tenant>`** is where agents coordinate, through `Handoff` nodes.

FalkorDB's multigraph model made the second and third trivial. Isolating a user or tenant is just a different graph name inside the same instance, with no extra deployments.

Tools are queries

Every tool the agent can call is a Cypher query or a graph algorithm:

Tool	What it does
`blast_radius`	Variable-length traversal over `DEPENDS_ON`
`recent_changes`	Walks upstream from a service to deploys and their authors
`who_knows`	Aggregates who resolved incidents on a service, plus its owning team
`decision_trace`	Follows a decision to its source document and flags superseded decisions
`critical_services`	`algo.pageRank` over the dependency graph
`path_between`	`shortestPath` between any two entities

A model with tool use picks which of these to call. The result is that the LLM does the planning and the graph does the knowing. If you remove FalkorDB the product has nothing to say, which is exactly the bar the hackathon sets.

Showing the path behind every answer

The part I care most about is explainability. Every tool call is recorded with its Cypher, its parameters and the rows it returned. The UI lists those steps and highlights the touched nodes on the graph, so an on-call engineer can see *why* the agent named a particular deploy rather than taking it on trust.

When the agent says "prime suspect: `deploy-4700`," you can click through to the query that found it.

Memory that stays current

Agent memory usually fails in one of two ways: it forgets, or it remembers stale facts forever. Hindsight stores facts as versioned nodes. When you teach it something new about an entity, the new `Fact` node gets a `SUPERSEDES` edge to the old one, and only the current fact is recalled by default. The history stays in the graph.

The same idea applies to company knowledge. In the seed data, `DEC-5` ("Restore Kafka retention to 7 days") supersedes `DEC-4` ("Cap Kafka retention at 48h"). The agent flags the older decision as stale instead of quoting it as policy.

Agents that coordinate through the graph

For a full incident, three agents run in sequence: Triage, Investigator and Scribe. They share no in-process state. Each one writes a `Handoff` node linked to a `Case`, and the next reads it from the graph. That makes the coordination inspectable and persistent, and any other agent could pick up the same case later.

What I learned

- **Model the questions first.** I sketched the questions an on-call engineer asks, then designed edges to answer them. Every relationship in the schema earns its place.
- **Algorithms are free tools.** PageRank and shortest path became agent tools with a few lines of Cypher each.
- **Versioning beats deleting.** `SUPERSEDES` edges gave me memory and knowledge freshness with one pattern.
- **Make it work offline.** Without an API key, a rule-based planner drives the same graph tools, so the demo never depends on an external service.

Honest limitations

The dataset is synthetic, as the rules require, so the next step is ingesting real incident and deploy data. The offline planner is a fixed chain rather than true reasoning. [ADD ANY MEASURED RESULTS HERE, such as query latency, once you have run it.]

Try it

```
docker compose up --build
```

Then open `localhost:8000`, ask *"What broke recently around postgres-main and who should I page?"*, and expand the trace.

Thanks to WeMakeDevs and FalkorDB for the hackathon.